FIAP

EcoTrack API

DevOps: pipeline de CI/CD, containerização e deploy em staging e produção

Aluno	Paulo Souza
RM	566228
Repositório	github.com/psouzah12/ecotrack-devops
Data	Outubro de 2026

1. Visão geral

O EcoTrack é a API REST que desenvolvi em Java 21 com Spring Boot para centralizar dados ambientais de empresas: cadastro de empresas, indicadores (água, energia, resíduos e CO2), ações ESG e relatórios. O banco é Oracle e o schema é versionado com Flyway.

Nesta entrega peguei o projeto da sprint anterior e montei em volta dele um pipeline de CI/CD que compila, testa, gera a imagem Docker e faz o deploy em staging e em produção, além de preparar a aplicação para rodar como em produção.

Repositório: github.com/psouzah12/ecotrack-devops
Execução documentada do pipeline: run #3

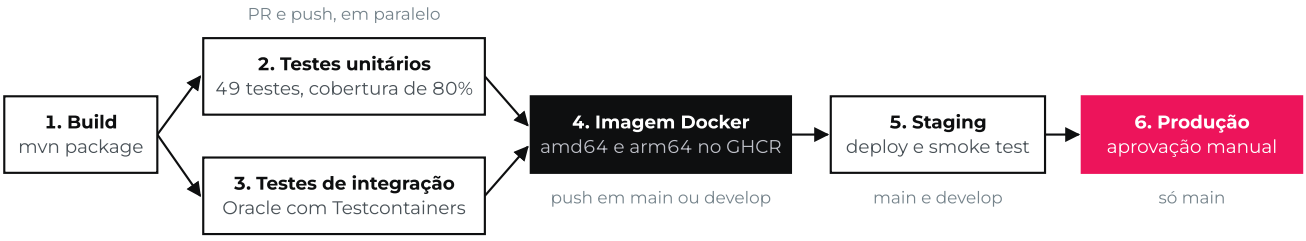
Pedido no enunciado	O que fiz
Pipeline com build, testes e deploy	GitHub Actions com 6 jobs: build, testes unitários, testes de integração, imagem no GHCR, deploy em staging e deploy em produção
Containerizar a aplicação	Dockerfile multi-stage com JRE 21 Alpine, usuário sem root, jar em camadas e healthcheck, com imagem para amd64 e arm64
Orquestrar a infraestrutura	docker-compose.yml com a API e o Oracle, redes frontend e backend (interna), volumes de dados e de logs e variáveis por ambiente
Simular um ambiente de produção	profiles de staging e produção, actuator, senhas fora do código, deploy com rollback, smoke test e banco de produção sem dados de teste

Mudanças na aplicação

- Coloquei o actuator: `/actuator/health` (com liveness e readiness, incluindo o banco) e `/actuator/info` com versão, commit e ambiente. O healthcheck do Docker e o smoke test usam esses dois.
- Criei os profiles `staging` e `prod`, com log em arquivo e rotação. Em produção o Flyway não aplica os dados de exemplo da pasta `db/seed`.
- Os usuários da API, que antes estavam fixos no `SecurityConfig`, agora vêm de variáveis de ambiente.
- JSON inválido, parâmetro com tipo errado, rota que não existe e método não suportado voltavam como 500. Agora voltam 400, 404 e 405, e o erro inesperado vai para o log.
- Desliguei o `open-in-view` e coloquei `@Transactional` nos services.
- Fiz uma página de status em `/`, com a identidade visual da FIAP e a fonte Montserrat, mostrando o ambiente, o health da API e do banco, a versão, o commit e o resumo ESG. O Swagger também mostra o ambiente no título.
- Os testes passaram de 2 para 56: 49 unitários e de controller e 7 de integração com Oracle.

2. Pipeline CI/CD

Usei o GitHub Actions. O workflow fica em `.github/workflows/ci-cd.yml` e a imagem vai para o GitHub Container Registry (GHCR), autenticado com o `GITHUB_TOKEN` do próprio run.



Job	O que faz
1. Build	Checkout, JDK 21 (Temurin) com cache do Maven e `mvn package`. O `ecotrack-api.jar` fica salvo como artefato.
2. Testes unitários	`mvn test` com 49 testes de services (Mockito) e de controllers com a segurança de verdade (MockMvc). Se a cobertura de linhas ficar abaixo de 80% o JaCoCo reprova o job.
3. Testes de integração	`mvn verify -DskipUnitTests`. O Testcontainers sobe um Oracle Free, o Flyway aplica as migrations e a API é testada via HTTP (CRUD, 401, 409 por chave estrangeira, health e info).
4. Imagem Docker	Buildx e QEMU geram a imagem para `linux/amd64` e `linux/arm64` e publicam no GHCR com as tags `sha-<commit>`, `1.1.0` e `latest`. O cache das camadas fica no próprio GitHub.
5. Deploy staging	Environment `staging`: login no GHCR, `scripts/deploy.sh staging <imagem>` e depois `scripts/smoke-test.sh`.
6. Deploy produção	Environment `production`, só no branch `main` e depois que staging passou. O Environment exige aprovação manual antes do job começar.

Como o pipeline funciona

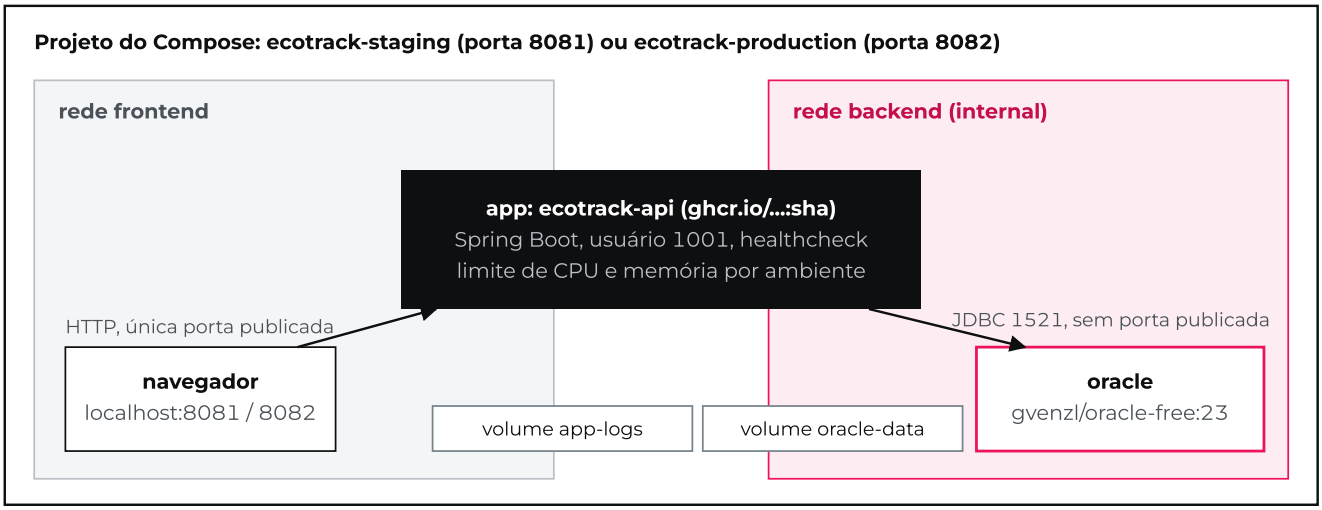
- Pull request para `main` roda só o build e os testes. Push em `develop` vai até staging e push em `main` vai até produção. Também dá para rodar manualmente.
- A imagem é gerada uma vez só, com a tag do commit. Staging e produção recebem a mesma imagem que passou nos testes.
- Se algum teste falhar, a imagem não é publicada, e produção só roda depois que staging passou no smoke test.
- O smoke test confere o health da API e do banco, se o ambiente e a versão no ar são os certos, a leitura pública, o bloqueio de escrita sem login e uma escrita real no banco (cria uma empresa e

apaga). O resultado aparece no resumo do run.

- Se a API não ficar healthy no tempo limite, o `deploy.sh` mostra os logs e volta para a imagem que estava rodando antes.
- As senhas ficam nos secrets de cada Environment do GitHub. Se não estiverem cadastradas, o `deploy.sh` gera senhas e guarda no servidor em `~/.ecotrack`, com permissão 600.
- Os deploys rodam no `ubuntu-24.04`: o ambiente inteiro sobe dentro do runner, passa pelo smoke test e é descartado no fim do job. Com a variável `DEPLOY_RUNNER=self-hosted`, os mesmos jobs fazem o deploy numa máquina própria.
- Execuções do mesmo branch esperam na fila (`cancel-in-progress: false`), então um deploy nunca é interrompido no meio. O Dependabot abre PRs de atualização que também passam pelo pipeline.

3. Docker

Arquitetura



Um único `docker-compose.yml` atende todos os ambientes; o que muda vem de variáveis (imagem, porta, profile, senhas e limites de recursos). Cada ambiente roda como um projeto do Compose separado, com containers, redes e volumes próprios. O Oracle fica só na rede `backend`, que é interna: ele não tem porta publicada nem acesso à internet, e só a API conversa com ele.

Recurso	Como usei
Serviços	`oracle` (gvenzl/oracle-free:23-slim) e `app`. A API só sobe depois que o Oracle fica healthy.
Redes	`frontend`, com a porta da API, e `backend`, interna, com a API e o Oracle.
Volumes	`oracle-data` com os dados do banco, que continuam entre deploys, e `app-logs` com o log da API.
Variáveis	`deploy/staging.env` e `deploy/production.env` sem senhas, secrets do GitHub para as senhas e o `.env.example` com todas as variáveis.
Operação	`restart: unless-stopped`, healthchecks, desligamento gracioso, limite de CPU e memória e rotação dos logs do Docker.

Dockerfile

```
# syntax=docker/dockerfile:1
# EcoTrack API: imagem em dois estágios. O primeiro compila com Maven e JDK 21
# e separa o jar em camadas; o segundo tem só o JRE 21 Alpine e roda sem root.

# O build roda na arquitetura de quem está construindo ($BUILDPLATFORM).
# O jar é o mesmo para amd64 e arm64, então o Maven executa uma vez só
# mesmo quando o pipeline gera a imagem multi-arquitetura.
FROM --platform=$BUILDPLATFORM maven:3.9.9-eclipse-temurin-21 AS build
WORKDIR /workspace
```

```
# 1) dependências primeiro: essa camada só é refeita quando o pom.xml muda
COPY pom.xml .
RUN mvn -B -ntp dependency:go-offline

# 2) código-fonte e empacotamento (os testes já rodaram nos jobs anteriores do pipeline)
COPY src ./src
RUN mvn -B -ntp package -DskipTests \
 && java -Djarmode=tools -jar target/ecotrack-api.jar extract --layers --launcher --destination
target/extracted


FROM eclipse-temurin:21-jre-alpine AS runtime

# usuário sem privilégios + pasta de logs (montada como volume no compose)
RUN addgroup -S ecotrack && adduser -S -G ecotrack -u 1001 ecotrack \
 && mkdir -p /app/logs && chown -R ecotrack:ecotrack /app
WORKDIR /app

# camadas da menos para a mais volátil: num deploy comum só "application" muda
COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/dependencies/ ./
COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/spring-boot-loader/ ./
COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/snapshot-dependencies/ ./
COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/application/ ./

# metadados preenchidos pelo pipeline (docker build --build-arg ...)
ARG APP_VERSION=dev
ARG GIT_COMMIT=local
ARG BUILD_DATE=unknown
LABEL org.opencontainers.image.title="EcoTrack API" \
      org.opencontainers.image.description="API REST de indicadores e ações ESG" \
      org.opencontainers.image.version="${APP_VERSION}" \
      org.opencontainers.image.revision="${GIT_COMMIT}" \
      org.opencontainers.image.created="${BUILD_DATE}" \
      org.opencontainers.image.authors="Paulo Souza - RM566228"

ENV APP_COMMIT=${GIT_COMMIT} \
    JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError -
Duser.timezone=America/Sao_Paulo -Djava.security.egd=file:/dev/./urandom"

USER ecotrack
EXPOSE 8080

# o Docker marca o container como unhealthy se a API ou o banco caírem
HEALTHCHECK --interval=15s --timeout=5s --start-period=90s --retries=5 \
  CMD wget -q -O /dev/null http://127.0.0.1:8080/actuator/health/readiness || exit 1

ENTRYPOINT ["java", "org.springframework.boot.loader.launch.JarLauncher"]
```

Imagem criada

Item	Valor
Nome e tags	`ghcr.io/psouzah12/ecotrack-api` com `sha-92e76ea`, `1.1.0` e `latest`, pública
Bases	`eclipse-temurin:21-jre-alpine` para rodar e `maven:3.9.9-eclipse-temurin-21` para o build
Plataformas	linux/amd64 e linux/arm64
Tamanho	408 MB; a imagem da sprint anterior tinha 707 MB
Camadas	dependências com cerca de 64 MB, loader com 4 KB e o código da aplicação com cerca de 0,3 MB
Segurança	usuário `ecotrack` (uid 1001), sem root; o contexto de build só leva o `pom.xml` e o `src/`

Comandos usados

```
# ambiente local (build, API e Oracle)
docker compose up --build -d
docker compose ps
docker compose logs -f app
docker compose down            # com -v apaga o volume do banco

# imagem
docker build -t ecotrack-api:local --build-arg APP_VERSION=1.1.0 --build-arg GIT_COMMIT=$(git rev-
parse HEAD) .
docker image inspect ecotrack-api:local
docker history ecotrack-api:local

# job 4 do pipeline: imagem para duas arquiteturas no GHCR
docker buildx build --platform linux/amd64,linux/arm64 -t ghcr.io/psouzah12/ecotrack-api:sha-92e76ea
--push .

# jobs 5 e 6: deploy de cada ambiente
scripts/deploy.sh staging ghcr.io/psouzah12/ecotrack-api:sha-92e76ea
scripts/smoke-test.sh http://localhost:8081 staging
scripts/deploy.sh production ghcr.io/psouzah12/ecotrack-api:sha-92e76ea
scripts/smoke-test.sh http://localhost:8082 production

# o que o deploy.sh roda por dentro
docker compose --env-file deploy/staging.env -p ecotrack-staging -f docker-compose.yml pull
docker compose --env-file deploy/staging.env -p ecotrack-staging -f docker-compose.yml up -d --wait -
-remove-orphans
```

02-docker-build (ecotrack-devops)

```
$ docker build -t ecotrack-api:local --build-arg APP_VERSION=1.1.0 --build-arg GIT_COMMIT=92e76ea5ffda0f5a00542f2721d994720389ec69 .
#1 [internal] load build definition from Dockerfile
#3 CACHED
#4 [internal] load metadata for docker.io/library/maven:3.9.9-eclipse-temurin-21
#5 [internal] load metadata for docker.io/library/eclipse-temurin:21-jre-alpine
#4 [internal] load metadata for docker.io/library/maven:3.9.9-eclipse-temurin-21
#6 [internal] load .dockerignore
#7 [build 1/6] FROM docker.io/library/maven:3.9.9-eclipse-temurin-21@sha256:3a4ab3276a087bf276f79cae96b1af04f53731bec53fb2e651aca79e4b10211e
#8 [runtime 1/7] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:51ab5e3302e7141ce665ca3ea85e8b5cd648eafbc3c0c90dd79d6537684e4555
#9 [internal] load build context
#10 [build 2/6] WORKDIR /workspace
#10 CACHED
#11 [build 3/6] COPY pom.xml .
#11 CACHED
#12 [build 4/6] RUN mvn -B -ntp dependency:go-offline
#12 CACHED
#13 [build 5/6] COPY src ./src
#14 [build 6/6] RUN mvn -B -ntp package -DskipTests  && java -Djarmode=tools -jar target/ecotrack-api.jar extract --layers --launcher --destination target/extract
ed
#14 DONE 11.4s
#15 [runtime 4/7] COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/dependencies/ ./
#15 CACHED
#16 [runtime 5/7] COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/spring-boot-loader/ ./
#16 CACHED
#17 [runtime 2/7] RUN addgroup -S ecotrack && adduser -S -G ecotrack -u 1001 ecotrack  && mkdir -p /app/logs && chown -R ecotrack:ecotrack /app
#17 CACHED
#18 [runtime 3/7] WORKDIR /app
#18 CACHED
#19 [runtime 6/7] COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/snapshot-dependencies/ ./
#19 CACHED
#20 [runtime 7/7] COPY --from=build --chown=ecotrack:ecotrack /workspace/target/extracted/application/ ./
#21 naming to docker.io/library/ecotrack-api:local done

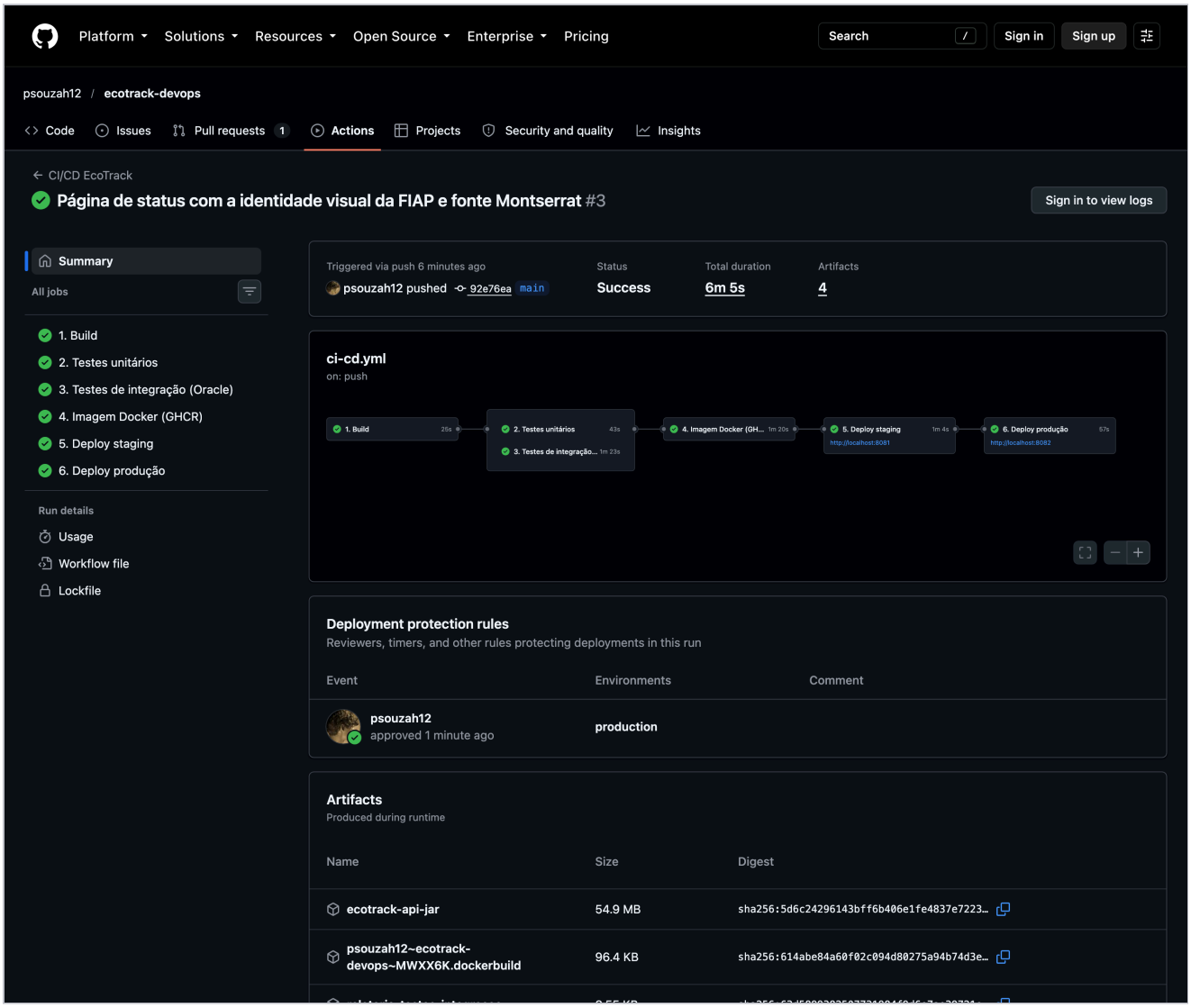
$ docker image ls ecotrack-api
REPOSITORY     TAG       IMAGE ID       SIZE
ecotrack-api   local     b93c1af6554f   408MB

$ docker image inspect ecotrack-api:local
Arquitetura: linux/arm64
Usuario:     ecotrack
Entrypoint:  ["java","org.springframework.boot.loader.launch.JarLauncher"]
Healthcheck: ["CMD-SHELL","wget -q -O /dev/null http://127.0.0.1:8080/actuator/health/readiness || exit 1"]
Porta:       {"8080/tcp":{}}
Versao:      1.1.0
Commit:      92e76ea5ffda0f5a00542f2721d994720389ec69

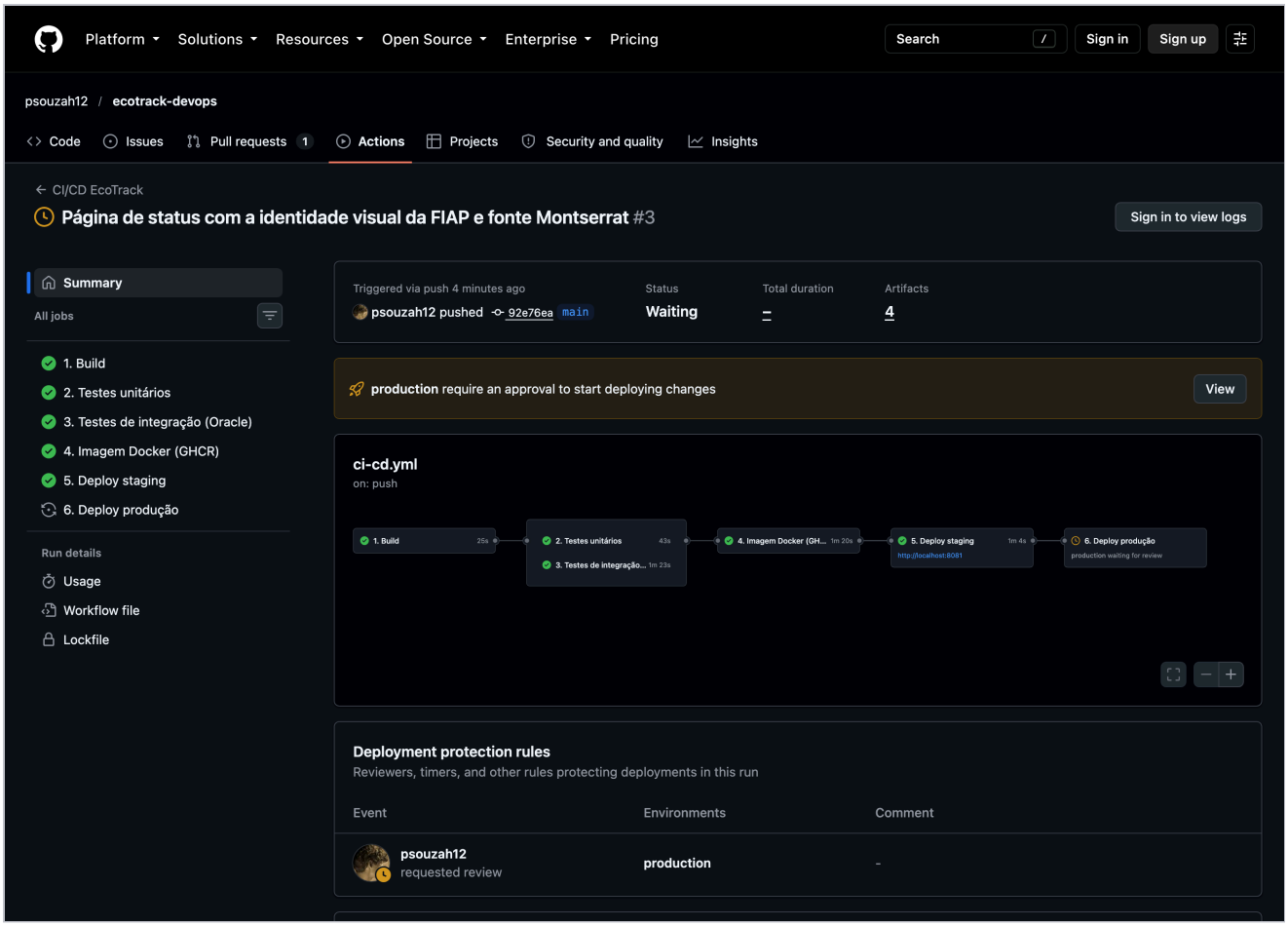
$ docker history ecotrack-api:local  (camadas)
SIZE      CREATED BY
0B        ENTRYPOINT ["java" "org.springframework.boot…
0B        HEALTHCHECK {Test:[CMD-SHELL wget -q -O /dev…
0B        EXPOSE [8080/tcp]
0B        USER ecotrack
0B        ENV APP_COMMIT=92e76ea5ffda0f5a00542f2721d99…
0B        LABEL org.opencontainers.image.title=EcoTrac…
0B        ARG BUILD_DATE=2026-10-14T01:12:49Z
0B        ARG GIT_COMMIT=92e76ea5ffda0f5a00542f2721d99…
0B        ARG APP_VERSION=1.1.0
532kB     COPY --chown=ecotrack:ecotrack /workspace/ta…
4.1kB     COPY --chown=ecotrack:ecotrack /workspace/ta…
688kB     COPY --chown=ecotrack:ecotrack /workspace/ta…
63.5MB    COPY --chown=ecotrack:ecotrack /workspace/ta…
```

Build local da imagem: estágios, camadas, usuário, healthcheck, versão e commit.

4. Pipeline em execução



Run #3 do workflow, disparado pelo push do commit 92e76ea em main. Os 6 jobs passaram em 6m05s, contando a espera pela aprovação de produção.



Depois do deploy em staging, o job de produção parou esperando a aprovação do Environment production.

GitHub Actions: log do job 1 (run 38493315480)

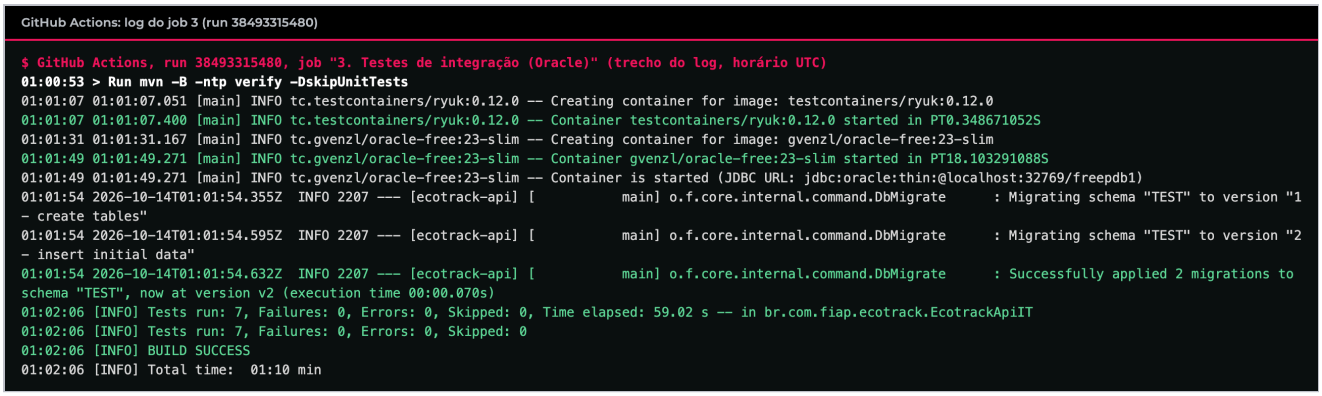
```
$ GitHub Actions, run 38493315480, job "1. Build" (trecho do log, horário UTC)
01:00:25 > Run mvn -B -ntp package -DskipTests
01:00:27 [INFO] Building EcoTrack API 1.1.0
01:00:40 [INFO] --- spring-boot:3.3.5:repackage (repackage) @ ecotrack-api ---
01:00:40 [INFO] BUILD SUCCESS
01:00:40 [INFO] Total time:  13.442 s
01:00:42 Artifact ecotrack-api-jar has been successfully uploaded! Final size is 57584338 bytes. Artifact ID is 11862896878
01:00:42 Artifact download URL: https://github.com/psouzah12/ecotrack-devops/actions/runs/38493315480/artifacts/11862896878
```

Job 1, build: trecho do log do GitHub Actions com o mvn package e o upload do jar.

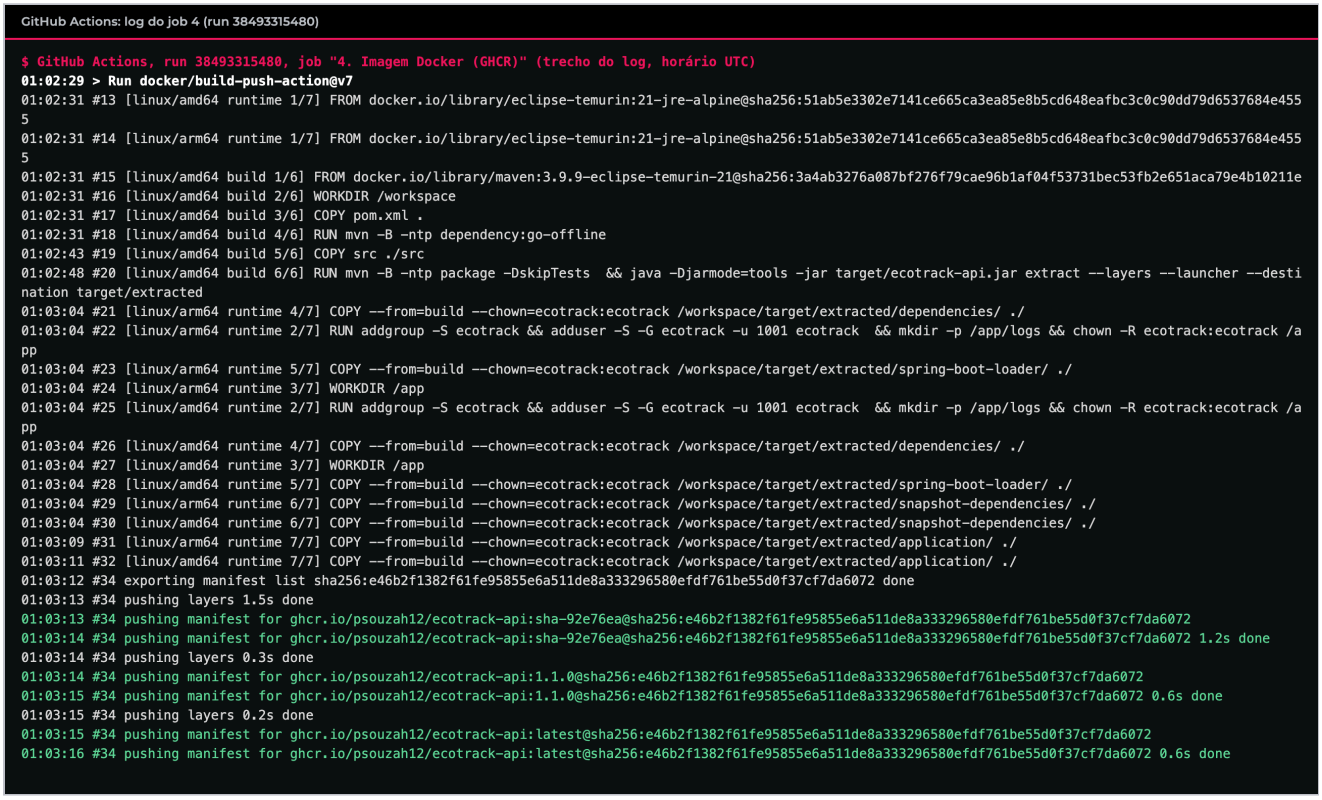
GitHub Actions: log do job 2 (run 38493315480)

```
$ GitHub Actions, run 38493315480, job "2. Testes unitários" (trecho do log, horário UTC)
01:00:56 > Run mvn -B -ntp test
01:01:18 [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.046 s -- in br.com.fiap.ecotrack.EcotrackApiApplicationTests
01:01:20 [INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.998 s -- in br.com.fiap.ecotrack.service.IndicadorServiceTest
01:01:20 [INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.141 s -- in br.com.fiap.ecotrack.service.AcaoServiceTest
01:01:20 [INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.098 s -- in br.com.fiap.ecotrack.service.EmpresaServiceTest
01:01:20 [INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.097 s -- in br.com.fiap.ecotrack.service.RelatorioServiceTest
01:01:24 [INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.946 s -- in br.com.fiap.ecotrack.controller.EmpresaControllerTest
01:01:25 [INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.658 s -- in br.com.fiap.ecotrack.controller.AcaoControllerTest
01:01:26 [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.697 s -- in br.com.fiap.ecotrack.controller.IndicadorControllerTest
01:01:26 [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.402 s -- in br.com.fiap.ecotrack.controller.RelatorioControllerTest
01:01:26 [INFO] Tests run: 49, Failures: 0, Errors: 0, Skipped: 0
01:01:26 [INFO] All coverage checks have been met.
01:01:26 [INFO] BUILD SUCCESS
01:01:26 [INFO] Total time:  19.415 s
```

Job 2, testes unitários: 49 testes sem falhas e cobertura mínima atingida.



Job 3, testes de integração: o Testcontainers sobe o Oracle, o Flyway aplica as migrations e os 7 testes passam.



Job 4, imagem Docker: o Maven roda só em linux/amd64, o estágio final é montado para amd64 e arm64 e as tags sha-92e76ea, 1.1.0 e latest vão para o GHCR.

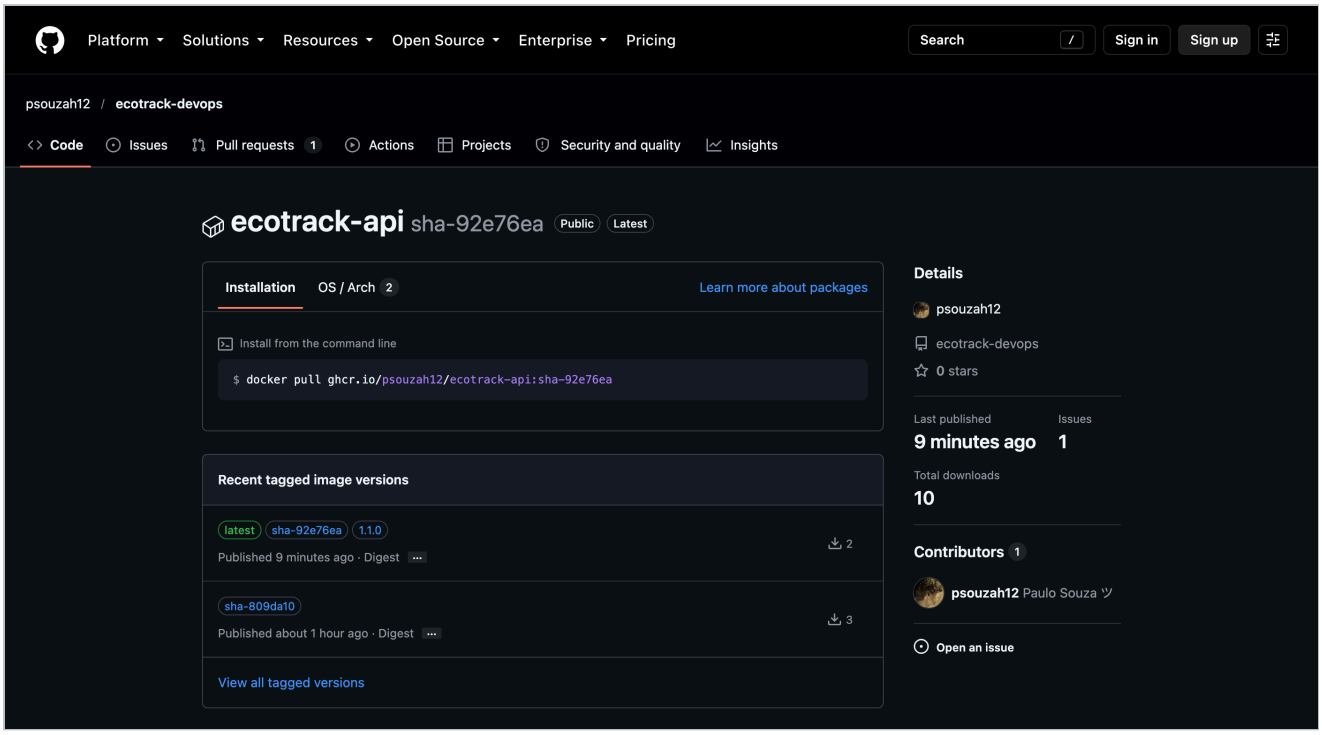
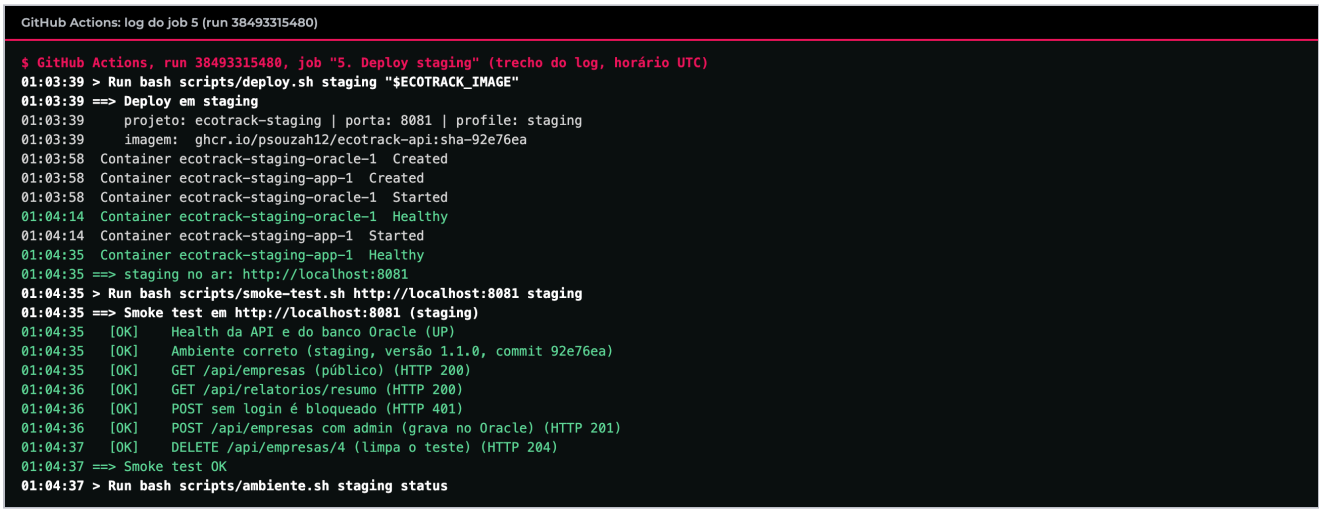


Imagem publicada no GitHub Container Registry, pública e com as duas arquiteturas.

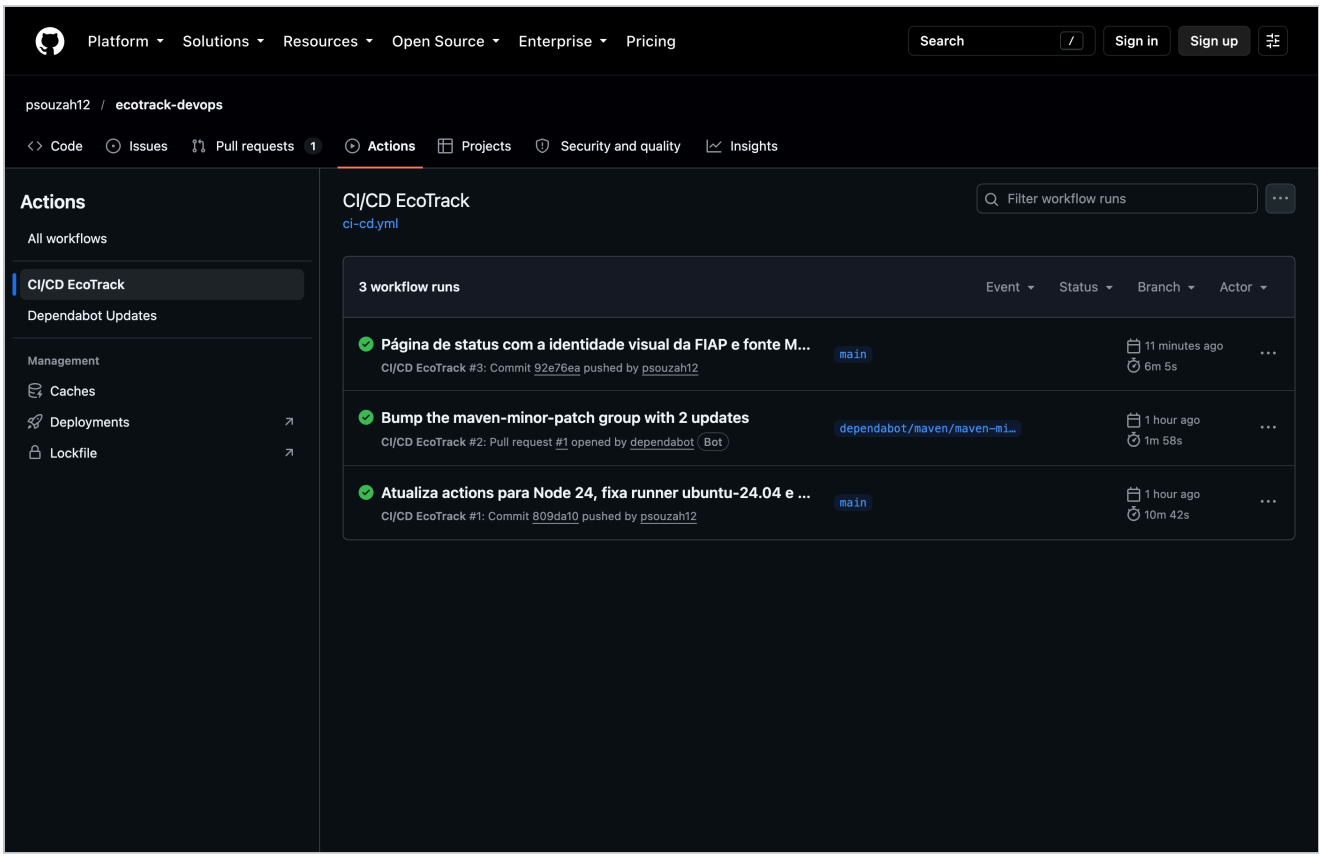


Job 5, deploy em staging com a imagem sha-92e76ea e smoke test com as 7 verificações OK.

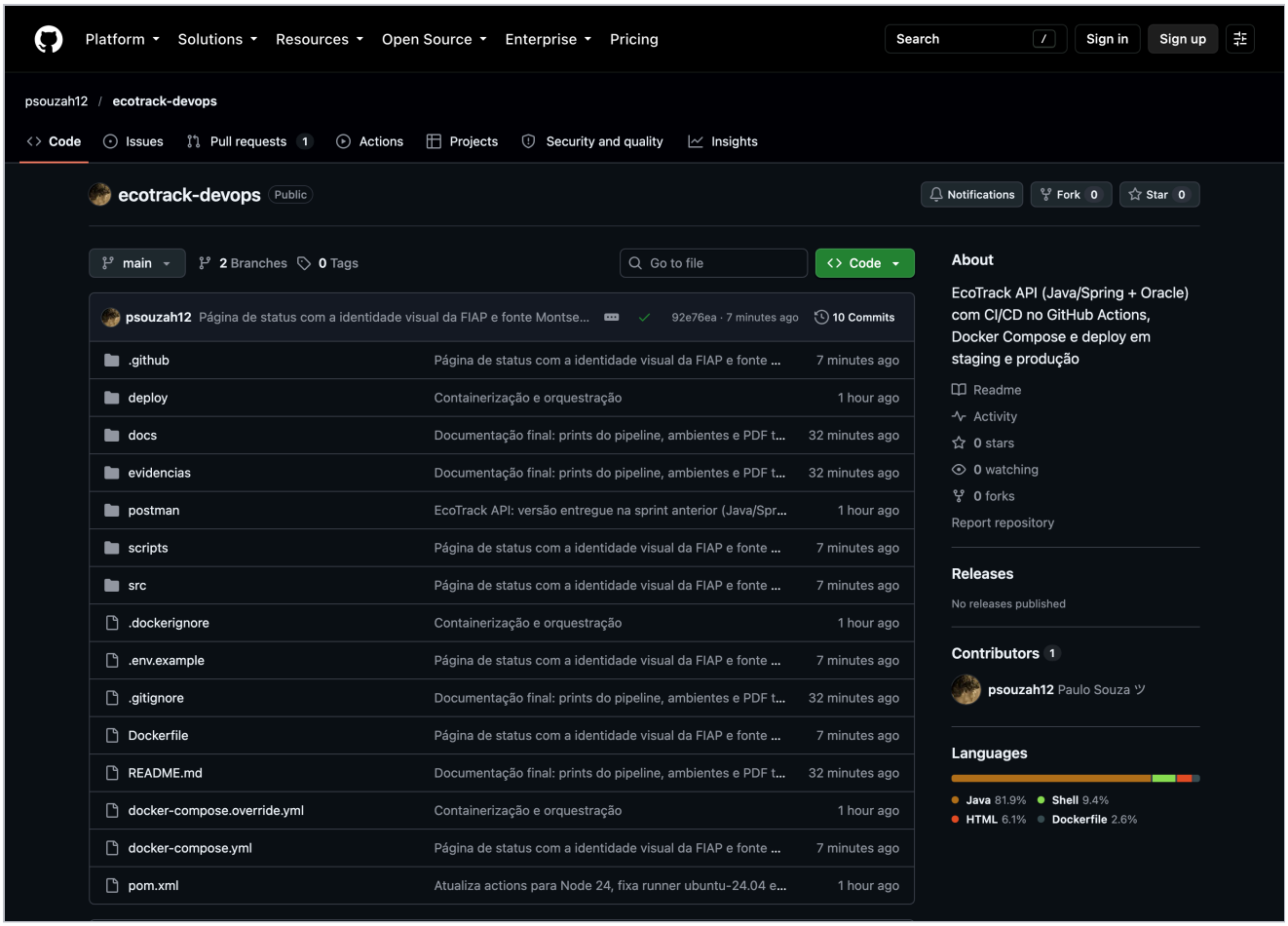
GitHub Actions: log do job 6 (run 38493315480)

```
$ GitHub Actions, run 38493315480, job "6. Deploy produção" (trecho do log, horário UTC)
01:05:29 > Run bash scripts/deploy.sh production "$ECOTRACK_IMAGE"
01:05:29 ==> Deploy em production
01:05:29     projeto: ecotrack-production | porta: 8082 | profile: prod
01:05:29     imagem:  ghcr.io/psouzah12/ecotrack-api:sha-92e76ea
01:05:47  Container ecotrack-production-oracle-1  Created
01:05:47  Container ecotrack-production-app-1  Created
01:05:47  Container ecotrack-production-oracle-1  Started
01:06:02  Container ecotrack-production-oracle-1  Healthy
01:06:02  Container ecotrack-production-app-1  Started
01:06:18  Container ecotrack-production-app-1  Healthy
01:06:18 ==> production no ar: http://localhost:8082
01:06:18 > Run bash scripts/smoke-test.sh http://localhost:8082 production
01:06:18 ==> Smoke test em http://localhost:8082 (production)
01:06:18   [OK]    Health da API e do banco Oracle (UP)
01:06:18   [OK]    Ambiente correto (producao, versão 1.1.0, commit 92e76ea)
01:06:18   [OK]    GET /api/empresas (público) (HTTP 200)
01:06:19   [OK]    GET /api/relatorios/resumo (HTTP 200)
01:06:19   [OK]    POST sem login é bloqueado (HTTP 401)
01:06:19   [OK]    POST /api/empresas com admin (grava no Oracle) (HTTP 201)
01:06:19   [OK]    DELETE /api/empresas/1 (limpa o teste) (HTTP 204)
01:06:19 ==> Smoke test OK
01:06:19 > Run bash scripts/ambiente.sh production status
```

Job 6, deploy em produção depois da aprovação, com a mesma imagem, os mesmos scripts e o smoke test OK.



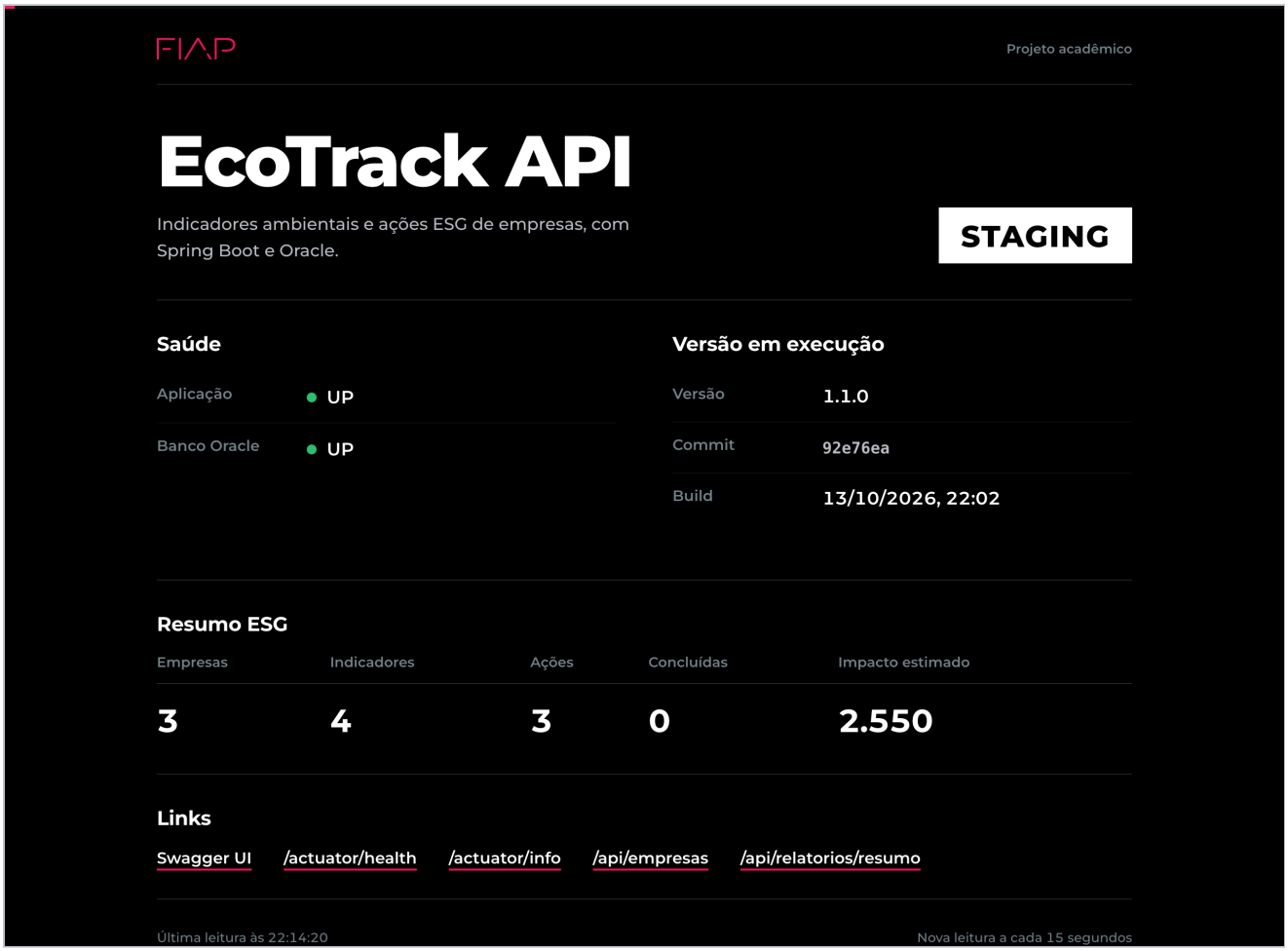
Histórico do workflow: os pushes em main e o PR do Dependabot, que também passa pelo pipeline antes de qualquer merge.



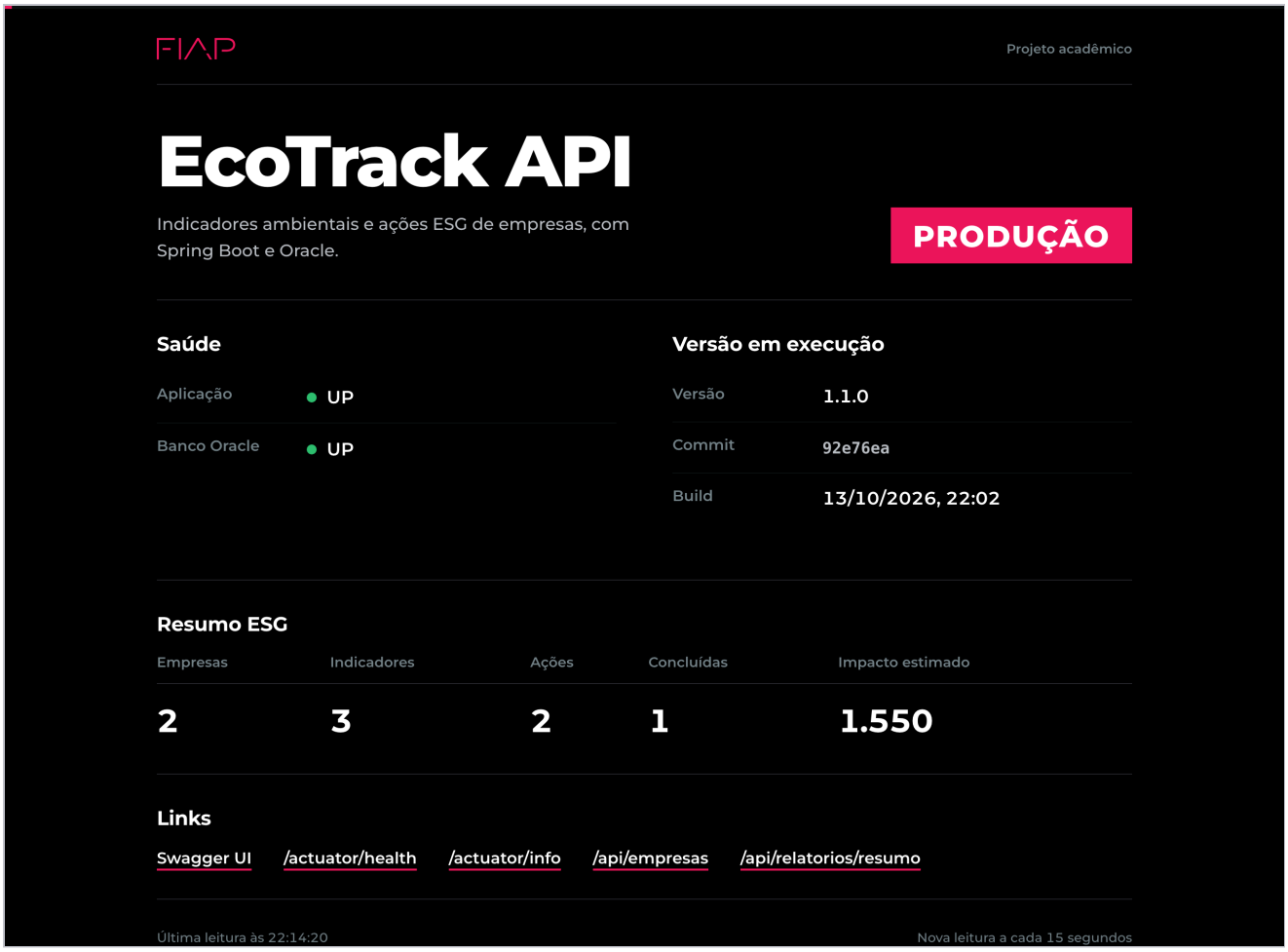
Repositório público com o código, o workflow e as evidências.

5. Staging e produção funcionando

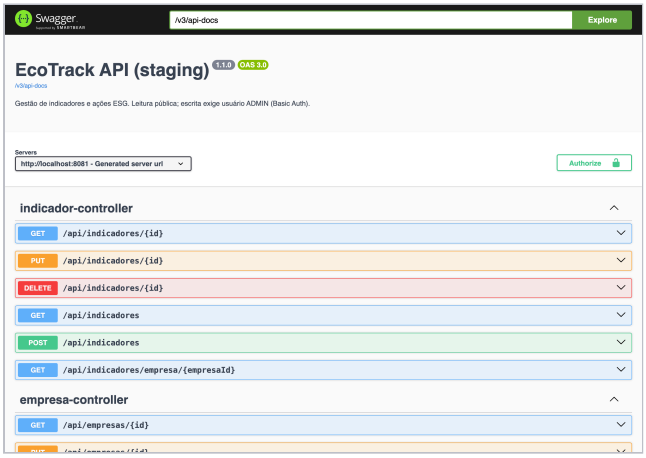
No pipeline, cada deploy sobe o ambiente dentro do runner do GitHub, passa pelo smoke test e é descartado no fim (seção 4). Para deixar os dois ambientes no ar, rodei os mesmos scripts na minha máquina com a imagem que o pipeline publicou, `ghcr.io/psouzah12/ecotrack-api:sha-92e76ea`. Staging tem os dados de exemplo. Produção começou vazia; cadastrei os registros pela API com o usuário admin e eles continuaram no volume do Oracle depois dos redeploys.



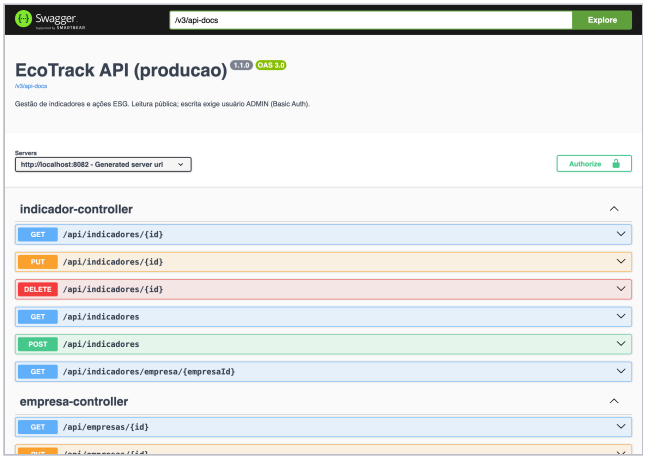
Página de status de staging (porta 8081): imagem sha-92e76ea, API e Oracle UP e dados de exemplo.



Página de status de produção (porta 8082): mesma imagem e mesmo commit, com banco próprio.



Swagger de staging.



Swagger de produção.

03-deploy-staging (ecotrack-devops)

```
$ scripts/deploy.sh staging ghcr.io/psouzah12/ecotrack-api:sha-92e76ea
==> Deploy em staging
    projeto: ecotrack-staging | porta: 8081 | profile: staging
    imagem:  ghcr.io/psouzah12/ecotrack-api:sha-92e76ea
    imagem em execução antes do deploy: ghcr.io/psouzah12/ecotrack-api:sha-809da10
 Container ecotrack-staging-oracle-1 Running
 Container ecotrack-staging-app-1 Recreate
 Container ecotrack-staging-app-1 Recreated
 Container ecotrack-staging-oracle-1 Waiting
 Container ecotrack-staging-oracle-1 Healthy
 Container ecotrack-staging-app-1 Starting
 Container ecotrack-staging-app-1 Started
 Container ecotrack-staging-oracle-1 Waiting
 Container ecotrack-staging-app-1 Waiting
 Container ecotrack-staging-oracle-1 Healthy
 Container ecotrack-staging-app-1 Healthy
==> staging no ar: http://localhost:8081
NAME                      IMAGE                                       COMMAND                  SERVICE   CREATED          STATUS                    PORTS
ecotrack-staging-app-1    ghcr.io/psouzah12/ecotrack-api:sha-92e76ea   "java org.springfram…"   app       11 seconds ago   Up 10 seconds (healthy)   0.0.0.0:808
1->8080/tcp, [::]:8081->8080/tcp
ecotrack-staging-oracle-1   gvenzl/oracle-free:23-slim                  "container-entrypoin…"   oracle    2 hours ago      Up 2 hours (healthy)

$ scripts/smoke-test.sh http://localhost:8081 staging
==> Smoke test em http://localhost:8081 (staging)
  [OK]    Health da API e do banco Oracle (UP)
  [OK]    Ambiente correto (staging, versão 1.1.0, commit 92e76ea)
  [OK]    GET /api/empresas (público) (HTTP 200)
  [OK]    GET /api/relatorios/resumo (HTTP 200)
  [OK]    POST sem login é bloqueado (HTTP 401)
  [OK]    POST /api/empresas com admin (grava no Oracle) (HTTP 201)
  [OK]    DELETE /api/empresas/41 (limpa o teste) (HTTP 204)
==> Smoke test OK
```

Deploy de staging com o scripts/deploy.sh e validação com o scripts/smoke-test.sh.

04-deploy-production (ecotrack-devops)

```
$ scripts/deploy.sh production ghcr.io/psouzah12/ecotrack-api:sha-92e76ea
==> Deploy em production
    projeto: ecotrack-production | porta: 8082 | profile: prod
    imagem:  ghcr.io/psouzah12/ecotrack-api:sha-92e76ea
    imagem em execução antes do deploy: ghcr.io/psouzah12/ecotrack-api:sha-809da10
 Container ecotrack-production-oracle-1 Running
 Container ecotrack-production-app-1 Recreate
 Container ecotrack-production-app-1 Recreated
 Container ecotrack-production-oracle-1 Waiting
 Container ecotrack-production-oracle-1 Healthy
 Container ecotrack-production-app-1 Starting
 Container ecotrack-production-app-1 Started
 Container ecotrack-production-app-1 Waiting
 Container ecotrack-production-oracle-1 Waiting
 Container ecotrack-production-oracle-1 Healthy
 Container ecotrack-production-app-1 Healthy
==> production no ar: http://localhost:8082
NAME                         IMAGE                                       COMMAND                  SERVICE   CREATED          STATUS                    PORTS
ecotrack-production-app-1    ghcr.io/psouzah12/ecotrack-api:sha-92e76ea   "java org.springfram…"   app       12 seconds ago   Up 10 seconds (healthy)   0.0.0.0:
8082->8080/tcp, [::]:8082->8080/tcp
ecotrack-production-oracle-1   gvenzl/oracle-free:23-slim                  "container-entrypoin…"   oracle    2 hours ago      Up 2 hours (healthy)

$ scripts/smoke-test.sh http://localhost:8082 production
==> Smoke test em http://localhost:8082 (production)
  [OK]    Health da API e do banco Oracle (UP)
  [OK]    Ambiente correto (producao, versão 1.1.0, commit 92e76ea)
  [OK]    GET /api/empresas (público) (HTTP 200)
  [OK]    GET /api/relatorios/resumo (HTTP 200)
  [OK]    POST sem login é bloqueado (HTTP 401)
  [OK]    POST /api/empresas com admin (grava no Oracle) (HTTP 201)
  [OK]    DELETE /api/empresas/10 (limpa o teste) (HTTP 204)
==> Smoke test OK
```

Deploy de produção e smoke test.

05-ambientes (ecotrack-devops)

```
$ docker ps   (staging e producao)
NAMES                          IMAGE                                        STATUS                    PORTS
ecotrack-production-app-1      ghcr.io/psouzah12/ecotrack-api:sha-92e76ea   Up 11 seconds (healthy)   0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
ecotrack-staging-app-1         ghcr.io/psouzah12/ecotrack-api:sha-92e76ea   Up 24 seconds (healthy)   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
ecotrack-production-oracle-1   gvenzl/oracle-free:23-slim                   Up 2 hours (healthy)
ecotrack-staging-oracle-1      gvenzl/oracle-free:23-slim                   Up 2 hours (healthy)

$ docker volume ls   (volumes dos dois ambientes)
DRIVER    VOLUME NAME
local     ecotrack-production_app-logs
local     ecotrack-production_oracle-data
local     ecotrack-staging_app-logs
local     ecotrack-staging_oracle-data

$ docker network ls   (redes; backend é internal)
NAME                           DRIVER    INTERNAL
ecotrack-production_backend    bridge    true
ecotrack-production_frontend   bridge    false
ecotrack-staging_backend       bridge    true
ecotrack-staging_frontend      bridge    false

$ docker stats --no-stream (consumo / limites)
NAME                           CPU %     MEM USAGE / LIMIT
ecotrack-production-app-1      0.35%     271.2MiB / 1GiB
ecotrack-staging-app-1         0.21%     270.6MiB / 768MiB
ecotrack-production-oracle-1   3.50%     2.375GiB / 7.75GiB
ecotrack-staging-oracle-1      2.57%     1.435GiB / 7.75GiB

$ curl -s http://localhost:8081/actuator/health
{"status":"UP","components":{"db":{"status":"UP"},"diskSpace":{"status":"UP"},"livenessState":{"status":"UP"},"ping":{"status":"UP"},"readinessState":{"status":"U
P"}},"groups":["liveness","readiness"]}

$ curl -s http://localhost:8082/actuator/info
{"app":{"ambiente":"producao","nome":"EcoTrack API","descricao":"Gestao de indicadores e acoes ESG","versao":"1.1.0","commit":"92e76ea5ffda0f5a00542f2721d99472038
9ec69"},"build":{"artifact":"ecotrack-api","name":"EcoTrack API","ti


$ docker exec ecotrack-production-app-1 tail -n 3 /app/logs/ecotrack-prod.log
2026-10-13T22:13:29.632-03:00  INFO 1 --- [ecotrack-api] [http-nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring Dispatc
2026-10-13T22:13:29.633-03:00  INFO 1 --- [ecotrack-api] [http-nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispa
2026-10-13T22:13:29.634-03:00  INFO 1 --- [ecotrack-api] [http-nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initialization in
```

Os dois ambientes: containers healthy, volumes, redes (a backend é interna), uso de memória dentro do limite e o log da API gravado no volume.

01-testes-unitarios (ecotrack-devops)

```
$ docker run --rm -v "$PWD":/workspace -w /workspace maven:3.9.9-eclipse-temurin-21 mvn -B test
[INFO] Building EcoTrack API 1.1.0
[INFO] --- jacoco:0.8.15:prepare-agent (prepare-agent) @ ecotrack-api ---
[INFO] argLine set to -javaagent:/root/.m2/repository/org/jacoco/org.jacoco.agent/0.8.15/org.jacoco.agent-0.8.15-runtime.jar=destfile=/workspace/target/jacoco.exe
c,excludes=br/com/fiap/ecotrack/EcotrackApiApplication.class
[INFO] Running br.com.fiap.ecotrack.controller.AcaoControllerTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.286 s -- in br.com.fiap.ecotrack.controller.AcaoControllerTest
[INFO] Running br.com.fiap.ecotrack.controller.EmpresaControllerTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.058 s -- in br.com.fiap.ecotrack.controller.EmpresaControllerTest
[INFO] Running br.com.fiap.ecotrack.controller.RelatorioControllerTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.364 s -- in br.com.fiap.ecotrack.controller.RelatorioControllerTest
[INFO] Running br.com.fiap.ecotrack.controller.IndicadorControllerTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.640 s -- in br.com.fiap.ecotrack.controller.IndicadorControllerTest
[INFO] Running br.com.fiap.ecotrack.EcotrackApiApplicationTests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in br.com.fiap.ecotrack.EcotrackApiApplicationTests
[INFO] Running br.com.fiap.ecotrack.service.AcaoServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.182 s -- in br.com.fiap.ecotrack.service.AcaoServiceTest
[INFO] Running br.com.fiap.ecotrack.service.EmpresaServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.051 s -- in br.com.fiap.ecotrack.service.EmpresaServiceTest
[INFO] Running br.com.fiap.ecotrack.service.IndicadorServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.047 s -- in br.com.fiap.ecotrack.service.IndicadorServiceTest
[INFO] Running br.com.fiap.ecotrack.service.RelatorioServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in br.com.fiap.ecotrack.service.RelatorioServiceTest
[INFO] Tests run: 49, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.15:report (report) @ ecotrack-api ---
[INFO] Loading execution data file /workspace/target/jacoco.exec
[INFO] --- jacoco:0.8.15:check (check) @ ecotrack-api ---
[INFO] Loading execution data file /workspace/target/jacoco.exec
[INFO] All coverage checks have been met.
[INFO] BUILD SUCCESS
[INFO] Total time:  8.869 s
```

Testes unitários rodando local com o mesmo JDK 21 do pipeline.

6. Desafios encontrados

Testcontainers e Docker 29

O teste de integração não achava o Docker e dava *BadRequestException* com status 400. O Docker 29 recusa clientes com API anterior à 1.44, e o Testcontainers 1.19.8 que vem com o Spring Boot 3.3.5 usa uma versão mais antiga. Resolvi fixando `testcontainers.version=1.21.4` no `pom.xml`.

LazyInitializationException com o open-in-view desligado

Quando desliguei o `spring.jpa.open-in-view`, as listagens e o PATCH de status passaram a dar 500, porque a conversão para DTO lia a empresa (LAZY) fora da sessão. Os testes unitários não pegaram isso; o teste de integração com Oracle pegou. Coloquei `@Transactional(readOnly = true)` nos services, `@Transactional` nos métodos de escrita e um teste de integração só para as listagens.

Erros 4xx viravam 500

O `@ExceptionHandler(Exception.class)` pegava tudo: rota inexistente, JSON mal formado e enum inválido voltavam como erro interno, sem nada no log. Criei handlers para 400, 404 e 405, coloquei log no handler genérico e testei cada caso.

Espaço no caminho do projeto

O projeto fica dentro de `Projeto ESG/`, e o comando do Compose guardado numa variável (`$COMPOSE up`) quebrava no espaço. Troquei a variável por uma função em bash com os argumentos entre aspas.

Disco cheio no meio do deploy

Ao subir o Oracle de staging o disco do Mac chegou a 100% e o Docker travou com erro de entrada e saída. O Docker Desktop guarda tudo num disco virtual que cresce, e cada Oracle ocupa uns 3 GB. Reiniciei o Docker, apaguei imagens antigas de outros projetos e o cache de build (uns 10 GB) e só depois subi os dois ambientes.

Senha do Oracle só vale na criação do volume

A imagem do Oracle cria o usuário da aplicação só no primeiro start. Se a senha mudasse a cada deploy, a API perderia o acesso ao banco. Por isso o `deploy.sh` reaproveita a senha: ela vem dos secrets do GitHub ou é gerada uma vez e guardada no servidor.

Mac arm64 e runners amd64

A imagem precisava rodar no meu Mac (arm64) e nos runners do GitHub (amd64). Usei Buildx com QEMU para gerar as duas arquiteturas, com o estágio do Maven em `$BUILDPLATFORM` para o Java ser compilado uma vez só e sem emulação.

Cache do Maven no build da imagem

Testei um `RUN --mount=type=cache` para o `~/.m2`, mas o cache do GitHub Actions não guarda esse tipo de cache, e o pipeline baixaria todas as dependências em toda execução. Deixei o `dependency:go-offline` numa camada própria, que entra no cache do GitHub e só é refeita quando o `pom.xml` muda.

O primeiro push não disparou o pipeline

O `paths-ignore` evita deploy quando só a documentação muda. No primeiro push de um branch novo, o GitHub comparou só o último commit, que mexia apenas no README e nas evidências. Rodei a primeira execução manualmente e, a partir daí, os pushes com código passaram a disparar o pipeline sozinhos.

Avisos de depreciação

A primeira execução passou, mas com 9 avisos: as actions `checkout@v4`, `setup-java@v4` e outras ainda rodavam em Node 20, que saiu dos runners. Atualizei as actions para as versões em Node 24, conferindo nas notas de versão que nenhuma opção que eu uso tinha mudado, e fixei o runner em `ubuntu-24.04`.

Dependabot e versões major

Logo depois do primeiro push o Dependabot abriu 10 PRs, inclusive para Spring Boot 4, springdoc 3 e Java 25. O pipeline rodou em cada um e reprovou o Spring Boot 4 e o springdoc 3. Configurei o Dependabot para ignorar versões major, que precisam de migração, e para agrupar as atualizações menores.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	✅
Dockerfile funcional	✅
docker-compose.yml ou arquivos Kubernetes	✅
Pipeline com etapas de build, teste e deploy	✅
README.md com instruções e prints	✅
Documentação técnica com evidências (PDF ou PPT)	✅
Deploy realizado nos ambientes staging e produção	✅